Geração do Conjunto de Mandelbrot usando Pthreads

Relatório de Programação Concorrente

Aluno: Ednardo Ferreira de Miranda
Profª: Raquel Coelho Gomes Pinto
Instituto Militar de Engenharia – IME

12 de outubro de 2026

Resumo

Este relatório descreve a implementação sequencial e paralela do conjunto de Mandelbrot em linguagem C. A versão paralela utiliza a biblioteca *Pthreads* e distribui o processamento da imagem entre múltiplas threads. Foram realizados experimentos no cluster LCAD2C do IME para imagens de 1000 × 1000, 2000 × 2000, 4000 × 4000 e 8000 × 8000 *pixels*, utilizando 1, 2, 4, 8, 16 e 32 threads. São apresentados os tempos médios, os *speedups*, as incertezas associadas e uma análise da escalabilidade observada.

1 Objetivo

O objetivo do trabalho é implementar duas versões para o cálculo do conjunto de Mandelbrot: uma sequencial e outra paralela com a biblioteca*Pthreads*. A versão paralela deve dividir o conjunto de pixels entre as threads, garantindo que todos os pixels sejam calculados uma única vez. Além disso, foram comparados os tempos de execução e o *speedup* obtido para diferentes tamanhos de imagem e números de *threads*.

A sequência utilizada para cada ponto complexo c foi:

$$z_0 = 0, \qquad z_{n+1} = z_n^2 + c. \tag{1}$$

O cálculo de um ponto foi interrompido quando:

$$|z_n| > 2 \tag{2}$$

ou quando foi atingido o limite de `MAX_ITER` = 1000 iterações.

2 Ambiente experimental

Os experimentos foram executados no Laboratório de Computação de Alto Desempenho para Defesa Cibernética (LCAD2C), utilizando o Sun Grid Engine (SGE) para submissão em batch. Os jobs foram submetidos por meio do comando `qsub -pe smp N`, sendo N o número de *threads* solicitado para a execução.

Conforme as informações fornecidas sobre o cluster, a arquitetura possui um frontend e três nós de computação: `compute-0-0`, `compute-0-1` e `compute-0-2`. Cada nó possui 80 núcleos a 2,10 GHz e 188 GB de memória RAM.

Foram realizadas três repetições para cada combinação de tamanho da imagem e quantidade de threads. O tempo medido correspondeu somente à região de processamento do conjunto de Mandelbrot. A alocação de memória, o cálculo do checksum e a impressão dos resultados ocorreram fora da região cronometrada.

3 Implementação

3.1 Versão sequencial

A versão sequencial percorre a imagem por linhas e colunas. Cada pixel (i, j) é convertido para um ponto do plano complexo conforme as equações:

$$c_x = x_{min} + j\frac{x_{max} - x_{min}}{largura - 1} \tag{3}$$

$$c_y = y_{min} + i\frac{y_{max} - y_{min}}{altura - 1}. \tag{4}$$

Foram utilizados os limites $x_{min} = -2,0$, $x_{max} = 1,0$, $y_{min} = -1,5$ e $y_{max} = 1,5$.

3.2 Versão paralela com Pthreads

A implementação Pthreads utiliza particionamento estático por linhas. Para p threads, a thread t recebe o intervalo de linhas:

$$inicio_t = \left\lfloor \frac{t \cdot altura}{p} \right\rfloor, \qquad fim_t = \left\lfloor \frac{(t+1) \cdot altura}{p} \right\rfloor. \tag{5}$$

Cada thread processa todas as colunas pertencentes às suas linhas. Dessa forma, não há sobreposição de escrita na imagem e não é necessário utilizar mutex. A thread principal utiliza `pthread_create()` para criar as threads e `pthread_join()` para esperar a conclusão de todas.

A correção foi validada por checksum. Para cada tamanho de imagem, o checksum permaneceu idêntico entre a versão sequencial, a versão Pthreads e todas as repetições.

4 Validação dos resultados

Tabela 1: Checksums obtidos para cada tamanho de imagem.

Imagem	Checksum
1000 × 1000	172299608
2000 × 2000	689833081
4000 × 4000	2760789602
8000 × 8000	11046094485

A igualdade dos checksums demonstra que a divisão por linhas não deixou pixels sem cálculo, não duplicou o cálculo de pixels e não introduziu condição de corrida na gravação da imagem.

5 Resultados experimentais

5.1 Tempos sequenciais

A Tabela 2 apresenta os tempos medidos para a versão sequencial. O desvio padrão (DP) foi calculado sobre as três repetições.

Tabela 2: Tempos da versão sequencial, em segundos.

Imagem	Rep. 1	Rep. 2	Rep. 3	Média T_{seq}	DP
1000 × 1000	1,126024	1,114083	1,115569	1,118558	0,006508
2000 × 2000	4,482404	5,091793	5,780504	5,118234	0,649454
4000 × 4000	17,924096	17,940162	17,930488	17,931582	0,008089
8000 × 8000	74,853632	73,851662	73,913249	74,206181	0,561554

5.2 Tempos paralelos

A Tabela 3 apresenta os tempos médios da implementação Pthreads. Cada célula apresenta média ± desvio padrão, em segundos.

Tabela 3: Tempo médio da versão Pthreads em segundos.

Imagem	1 thread	2 threads	4 threads	8 threads	16 threads	32 threads
1000 × 1000	1,116 ± 0,003	0,559 ± 0,001	0,543 ± 0,001	0,385 ± 0,000	0,229 ± 0,003	0,130 ± 0,000
2000 × 2000	4,468 ± 0,017	2,238 ± 0,000	2,171 ± 0,000	1,540 ± 0,007	0,930 ± 0,053	0,595 ± 0,019
4000 × 4000	18,186 ± 0,370	9,602 ± 0,647	8,834 ± 0,218	6,438 ± 0,502	3,840 ± 0,427	1,912 ± 0,002
8000 × 8000	73,484 ± 0,234	36,770 ± 0,643	35,937 ± 0,712	24,758 ± 0,408	14,342 ± 0,045	8,048 ± 0,718

5.3 Speedup

O speedup foi calculado conforme a Equação 6:

$$S(p) = \frac{T_{seq}}{T_p}, \tag{6}$$

onde T_{seq} é o tempo médio sequencial para o tamanho de imagem considerado e T_p é o tempo médio da versão paralela com p threads.

As barras de erro foram calculadas pela propagação dos desvios padrões dos tempos sequencial e paralelo:

$$\sigma_S = S\sqrt{\left(\frac{\sigma_{seq}}{T_{seq}}\right)^2 + \left(\frac{\sigma_p}{T_p}\right)^2}. \tag{7}$$

Tabela 4: Speedup médio e erro propagado.

Imagem	1 thread	2 threads	4 threads	8 threads	16 threads	32 threads
1000 × 1000	1,002 ± 0,006	2,000 ± 0,012	2,060 ± 0,012	2,905 ± 0,017	4,875 ± 0,068	8,593 ± 0,060
2000 × 2000	1,146 ± 0,145	2,287 ± 0,290	2,357 ± 0,299	3,323 ± 0,422	5,505 ± 0,766	8,601 ± 1,126
4000 × 4000	0,986 ± 0,020	1,868 ± 0,126	2,030 ± 0,050	2,785 ± 0,217	4,670 ± 0,519	9,376 ± 0,010
8000 × 8000	1,010 ± 0,008	2,018 ± 0,038	2,065 ± 0,044	2,997 ± 0,054	5,174 ± 0,042	9,220 ± 0,826

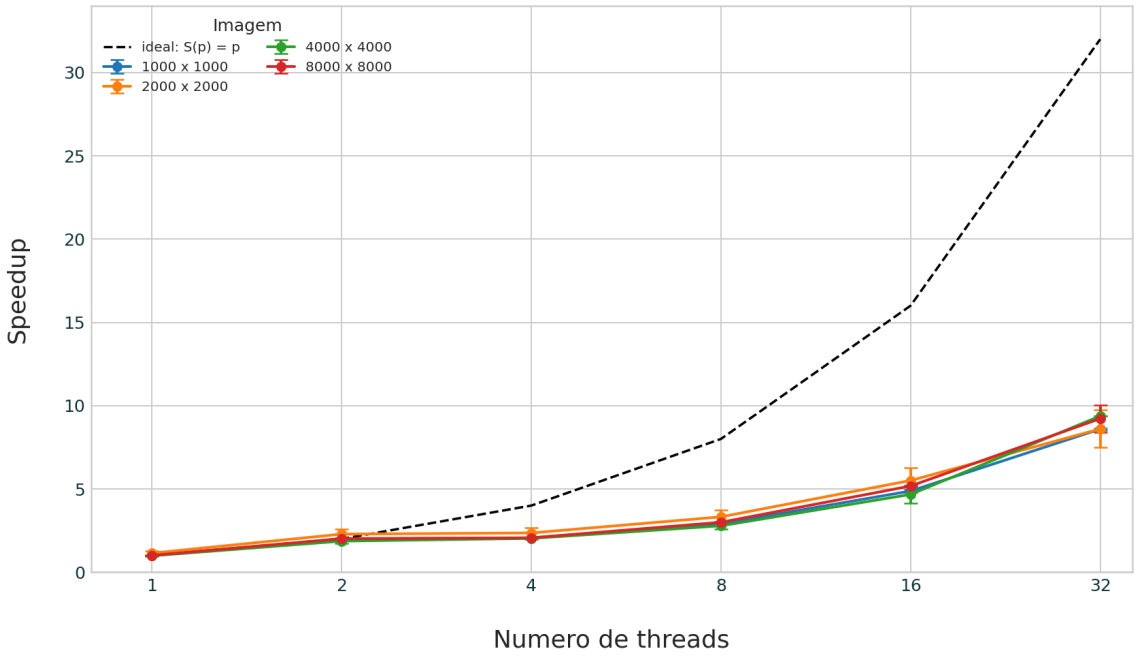


Figura 1: Speedup em função do número de threads. A reta tracejada representa o speedup ideal $S(p) = p$. As barras verticais representam o erro propagado a partir dos desvios padrões das medições.

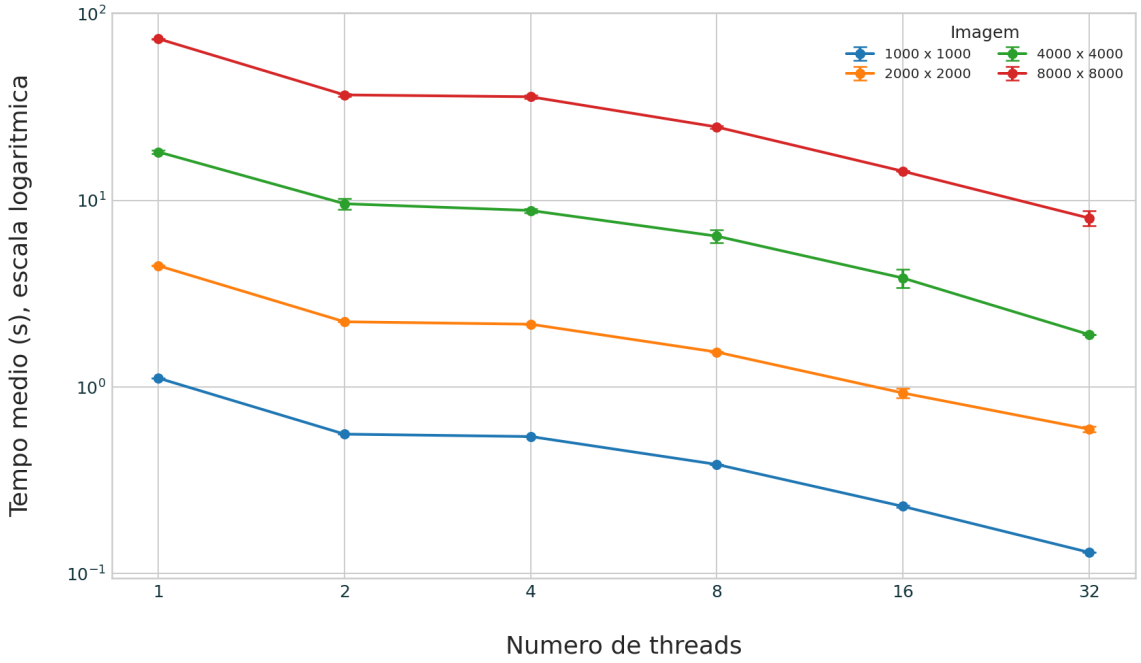


Figura 2: Tempo médio da versão Pthreads em função do número de threads. O eixo vertical está em escala logarítmica.

6 Discussão

6.1 Aumento do número de threads

Considerando os valores médios, o aumento do número de threads reduziu o tempo de execução em todos os tamanhos avaliados. Entretanto, o ganho não foi uniforme. A passagem de 2 para 4 threads apresentou ganho pequeno. Para 8000 × 8000, por exemplo, o tempo médio reduziu de 36,770 s para 35,937 s. Para 1000 × 1000, reduziu de 0,559 s para 0,543 s.

6.2 Comparação com o speedup ideal

O comportamento ideal seria $S(p) = p$. Os valores experimentais ficaram abaixo desse limite em todas as configurações. Com 32 threads, os maiores speedups foram 9,376 para 4000 × 4000 e 9,220 para 8000 × 8000. Esses valores representam aproximadamente 29,3% e 28,8% do speedup ideal de 32, respectivamente.

Para a imagem de 1000 × 1000, o speedup máximo foi 8,593, correspondente a aproximadamente 26,9% do ideal. Para 2000 × 2000, o valor foi 8,601, mas a incerteza foi maior em razão da dispersão nas medições sequenciais. Portanto, a implementação apresenta ganho discreto.

Todavia, o efeito não foi monotônico para todos os valores de p. A irregularidade do conjunto de Mandelbrot, a hierarquia de memória e a variação de carga dos nós também influenciaram os resultados. O tamanho 2000 × 2000 teve maior incerteza porque os tempos sequenciais apresentaram dispersão elevada.

6.3 Ponto de retorno decrescente

O primeiro ponto de retorno decrescente apareceu entre 2 e 4 threads. Para 4000 × 4000, o speedup aumentou de 1,868 para apenas 2,030. Para 8000 × 8000, aumentou de 2,018 para 2,065. Após 4 threads, os ganhos voltaram a ser significativos, mas continuaram muito inferiores ao comportamento linear ideal.

A passagem de 16 para 32 threads ainda gerou redução de tempo relevante. Para 8000 × 8000, o tempo caiu de 14,342 s para 8,048 s. Mesmo assim, a eficiência paralela com 32 threads permaneceu próxima de 29% para os maiores problemas, evidenciando limitações de escalabilidade.

7 Conclusão

A versão paralela com Pthreads foi validada pelos checksums e apresentou redução substancial do tempo de execução para todos os tamanhos de imagem. Com 32 threads, os speedups variaram entre 8,593 e 9,376. Embora os resultados estejam abaixo do speedup ideal, a implementação demonstrou ganho efetivo de desempenho.